

# Aproximación y caracterización de curvas paramétricas

Recorrido completo del proyecto: del EDA al sistema adaptativo y la reconstrucción desde píxeles

## Resumen

Este documento sigue paso a paso el trabajo realizado sobre un conjunto de 500 curvas paramétricas 2D. El recorrido empieza por entender qué son los datos (cinco notebooks de EDA), pasa por probar tres familias de modelos para aproximarlas (polinomios, suma de gaussianas y splines), continúa caracterizando geométricamente qué tipo de curva favorece a cada método, y termina construyendo un sistema adaptativo que recomienda automáticamente el ajuste óptimo y un experimento de reconstrucción que parte de versiones pixelizadas de las curvas. Todas las cifras y figuras provienen directamente de los notebooks del repositorio.

## Índice

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1. Punto de partida: los datos</b>                       | <b>2</b>  |
| <b>2. Fase 1: exploración de los datos</b>                  | <b>2</b>  |
| 2.1. Estructura general (notebook 01) . . . . .             | 2         |
| 2.2. Análisis estadístico (notebook 02) . . . . .           | 2         |
| 2.3. Análisis geométrico (notebook 03) . . . . .            | 3         |
| 2.4. Anomalías y galería (notebook 04) . . . . .            | 3         |
| 2.5. Curvas como funciones (notebook 05) . . . . .          | 4         |
| <b>3. Fase 2: aproximaciones</b>                            | <b>4</b>  |
| 3.1. Polinomios . . . . .                                   | 5         |
| 3.1.1. Primer intento (versión 1) . . . . .                 | 5         |
| 3.1.2. Versión 2: criterios profesionales . . . . .         | 5         |
| 3.2. Gaussianas . . . . .                                   | 6         |
| 3.2.1. Primer intento: una sola campana . . . . .           | 6         |
| 3.2.2. Suma de gaussianas y splines (notebook 02) . . . . . | 8         |
| <b>4. Fase 3: comparación entre métodos</b>                 | <b>9</b>  |
| <b>5. Fase 4: caracterización</b>                           | <b>10</b> |
| <b>6. Fase 5: sistema adaptativo</b>                        | <b>12</b> |
| 6.1. Selector de método . . . . .                           | 13        |
| 6.2. Segmentación adaptativa . . . . .                      | 13        |
| 6.3. Benchmark y síntesis . . . . .                         | 13        |
| <b>7. Fase 6: reconstrucción desde píxeles</b>              | <b>13</b> |
| 7.1. Caso individual . . . . .                              | 14        |
| 7.2. Reconstrucción masiva . . . . .                        | 14        |
| 7.3. Análisis de fallos . . . . .                           | 14        |
| 7.4. Modo interactivo . . . . .                             | 15        |
| <b>8. Conclusión global</b>                                 | <b>15</b> |

---

## 1. Punto de partida: los datos

El dataset contiene 500 curvas paramétricas, cada una representada en dos formatos. El primero, en `datos/target/`, son archivos de alta precisión con 8 000 puntos  $(x, y)$  por curva, valores flotantes. Es la verdad de referencia. El segundo, en `datos/pixel_curves/`, simula cómo se vería esa misma curva si se digitalizara a mano desde una imagen: cinco versiones por curva con coordenadas enteras a distintas escalas (X10, X15, X18, X21, X25), que producen entre 200 y 500 puntos según el factor.

El dominio de  $x$  va de aproximadamente 5 a 260 y los valores de  $y$  se mueven entre 0 y 150. El total son 3 000 archivos: 500 target más 2 500 pixelizadas. La integridad fue verificada celda por celda: ningún archivo vacío, ningún NaN, ningún infinito, y la correspondencia entre target y píxeles es perfecta.

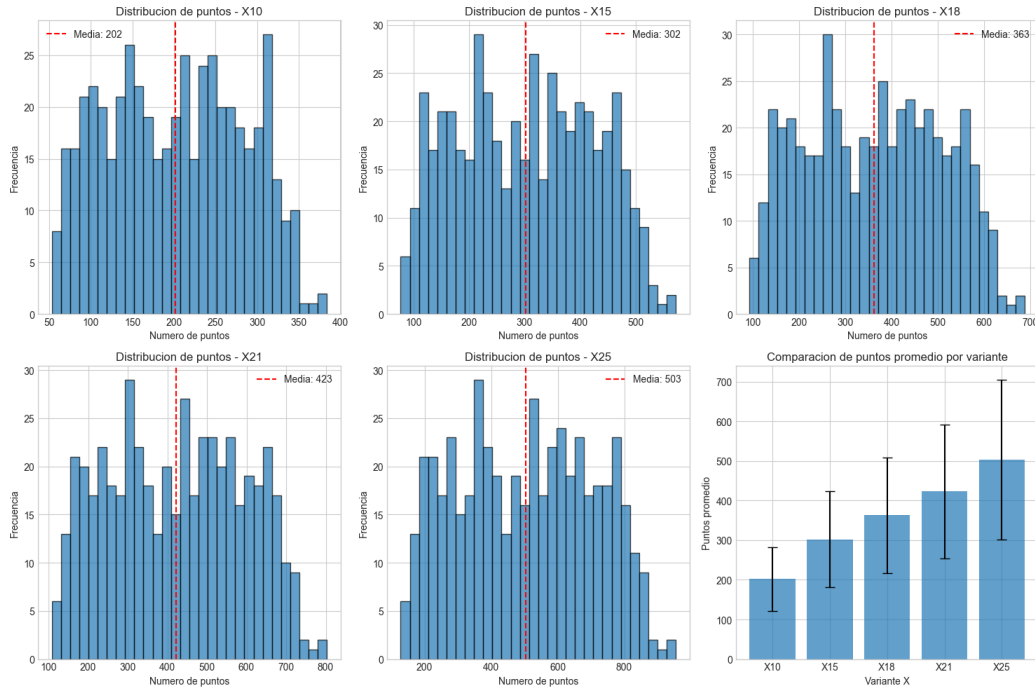


Figura 1: Vista resumen de la estructura del dataset (notebook 01 del EDA).

## 2. Fase 1: exploración de los datos

Antes de aproximar nada hubo que entender el material. La exploración se repartió en cinco notebooks dentro de `exploracion_datos/`. Cada uno aborda una pregunta distinta.

### 2.1. Estructura general (notebook 01)

Este primer notebook documenta los formatos, verifica la nomenclatura y comprueba la integridad de los archivos. Confirma que los 500 ID van de 0001 a 0500, que el target siempre tiene 8 000 puntos y que el número de puntos pixelizados depende linealmente del factor de escala.

### 2.2. Análisis estadístico (notebook 02)

Aquí se analizan las distribuciones de las coordenadas, el comportamiento del parámetro  $x$  y la relación entre target y píxeles. La fórmula empírica que ajusta el número de puntos pixelizados en función de la escala es aproximadamente  $\text{puntos} \approx 35,1 \cdot X - 63,9$ , con  $R^2$  muy alto.

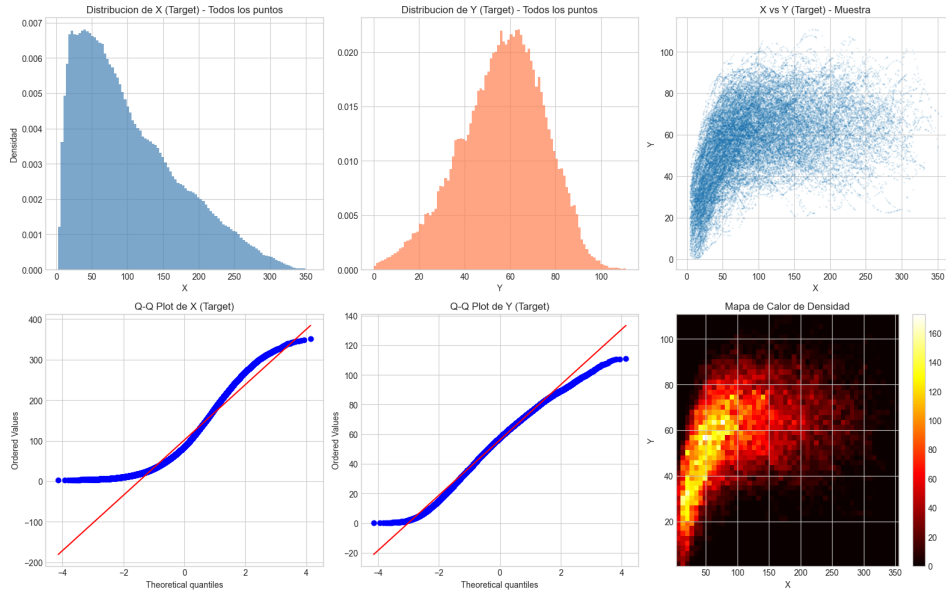


Figura 2: Distribuciones de coordenadas y métricas por curva (EDA 02).

### 2.3. Análisis geométrico (notebook 03)

El tercer notebook calcula longitud de arco, curvatura, pendientes y velocidades paramétricas. Aparecen los descriptores que después serán cruciales: número de máximos y mínimos locales, dispersión de la curvatura, y rango de pendientes. La mayoría de las curvas tienen velocidad paramétrica relativamente uniforme, lo que sugiere parametrización por longitud de arco o algo cercano.

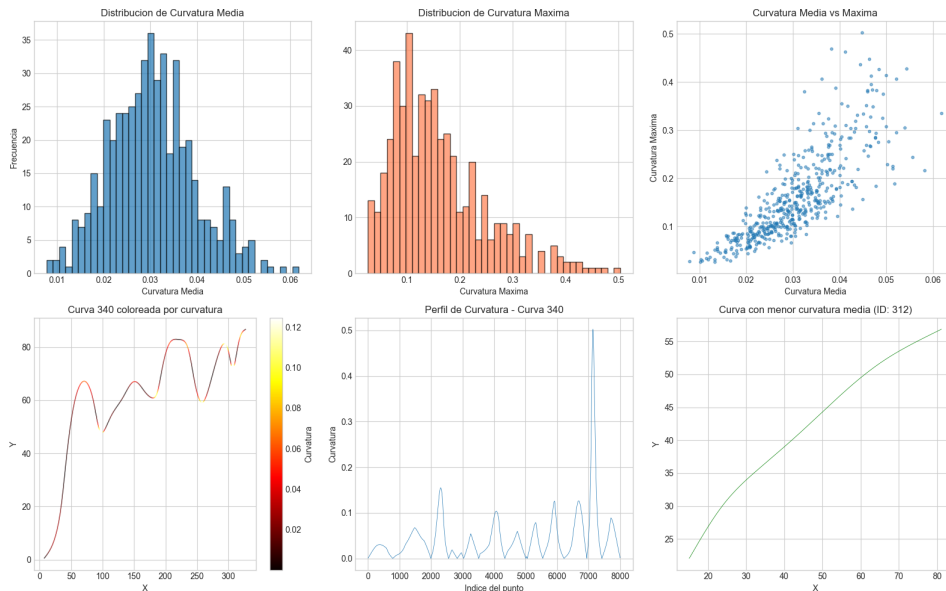


Figura 3: Distribuciones de curvatura y pendientes (EDA 03).

### 2.4. Anomalías y galería (notebook 04)

Se detectan outliers, se inspecciona la calidad de la discretización pixel por pixel y se construye una galería visual. No aparecen curvas patológicas: ninguna está vacía, ninguna se sale del rango esperado, ninguna tiene saltos sospechosos.

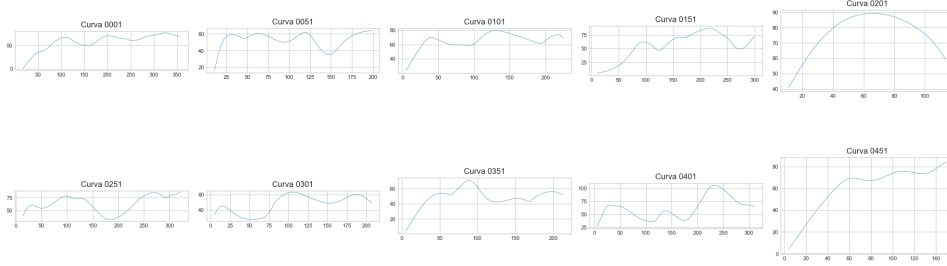


Figura 4: Galería representativa de curvas y métricas multi-resolución (EDA 04).

## 2.5. Curvas como funciones (notebook 05)

El quinto notebook responde la pregunta clave para el resto del proyecto: ¿son funciones  $y = f(x)$  bien definidas? La verificación es afirmativa para las 500: cada  $x$  tiene un único  $y$ , no hay discontinuidades, y existe estructura interpretable. A partir de aquí se puede tratar cada curva como una función de una variable y plantear ajustes funcionales.

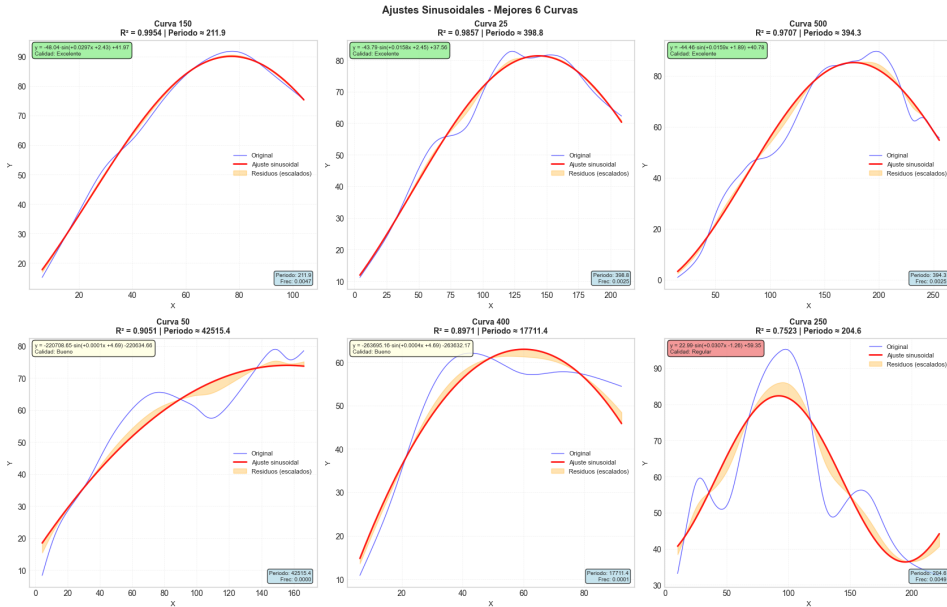


Figura 5: Tipos de funciones identificadas (EDA 05).

**Salida de la fase:** el archivo `exploracion_datos/caracteristicas_geometricas.csv` con 23 métricas geométricas para cada una de las 500 curvas (curvatura, pendiente, número de máximos y mínimos, longitud de arco, etcétera).

## 3. Fase 2: aproximaciones

Confirmado que las curvas son funciones, la siguiente pregunta es si pueden reconstruirse con un modelo matemático compacto. Se probaron tres familias.

### 3.1. Polinomios

#### 3.1.1. Primer intento (versión 1)

El intento más sencillo: ajustar un polinomio de grado  $d$  y aceptarlo si  $R^2 \geq 0,99$ . Funciona en muchas curvas, pero el criterio es naive: el  $R^2$  siempre sube al añadir parámetros, así que esta política tiende a sobreajustar.

#### 3.1.2. Versión 2: criterios profesionales

El segundo notebook (`aproximacion_polynomial_v2.ipynb`) reemplaza el criterio simple por una batería:  $R^2$  ajustado, AIC, BIC, validación cruzada y método del codo. Para cada una de las 500 curvas y cada una de las 5 escalas se prueban grados del 1 al 20 y se selecciona el óptimo según cada criterio.

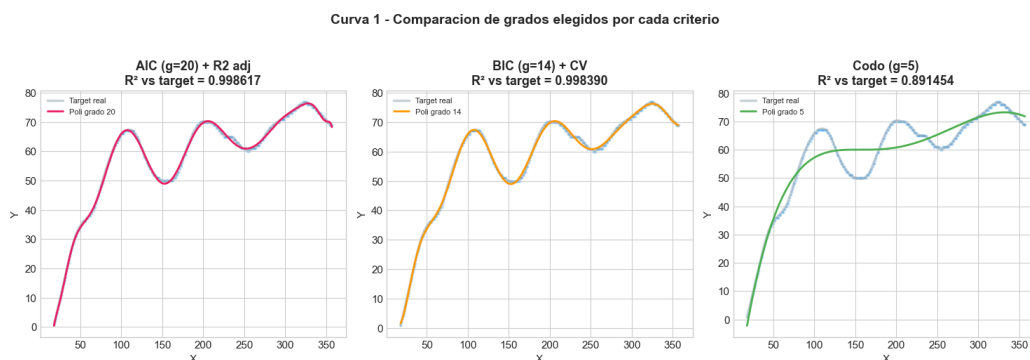


Figura 6: Comportamiento del  $R^2$  y los criterios de información en función del grado polinomial.

El resultado es sorprendentemente bueno: incluso desde la escala más baja (X10), los polinomios alcanzan  $R^2 > 0,97$  en promedio. El grado óptimo según BIC se concentra entre 8 y 10. Las cinco escalas producen resultados casi idénticos, indicando robustez al ruido de pixelización.

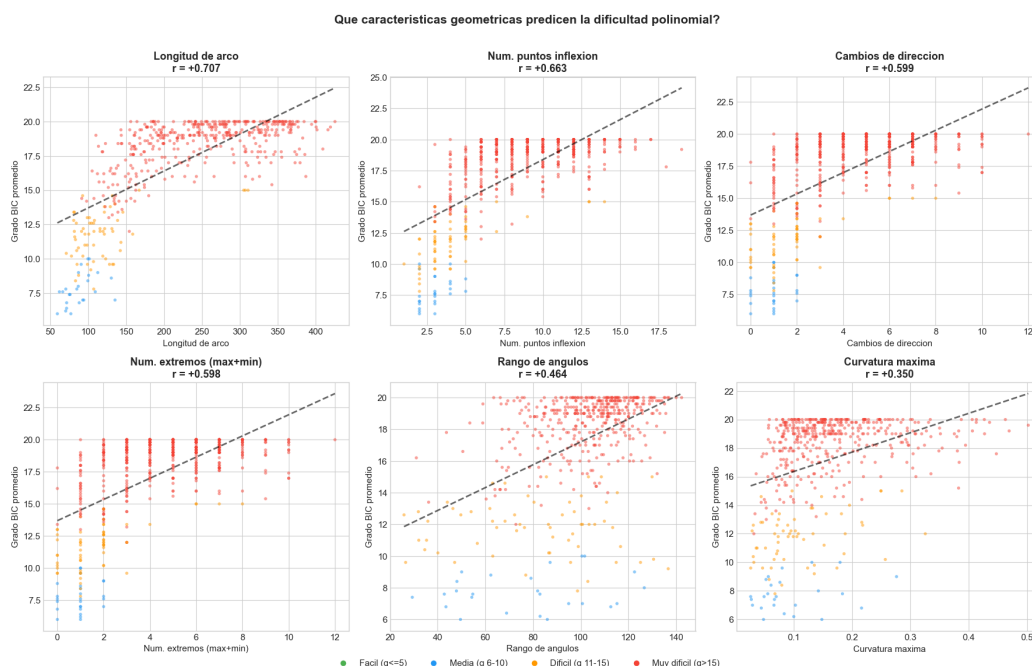


Figura 7: Distribución de grados óptimos (BIC) sobre las 500 curvas.

Sobre el target de alta precisión, el ajuste polinomial alcanza  $R^2$  medio de 0,998 y mediana 0,9996. Las salidas de esta fase son dos tablas: `grados_optimos_por_curva.csv` (500 filas, una por curva) y `detalle_criterios_por_curva_escalas.csv` (2 500 filas, una por combinación curva-escala).

## 3.2. Gaussianas

### 3.2.1. Primer intento: una sola campana

La motivación: si una curva tiene un pico marcado, quizás una gaussiana lo describe mejor que un polinomio. El modelo es

$$y = A \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}} + c.$$

Se ajusta por mínimos cuadrados no lineales con `scipy.optimize.curve_fit`.

# Ajuste Gaussiano — 1 campana por curva



Figura 8: Ajuste con una sola gaussiana sobre cinco curvas de ejemplo (notebook 01 de gaussianas).

El resultado promedio es  $R^2 \approx 0,74$ . Una sola campana no alcanza para curvas con varios máximos.

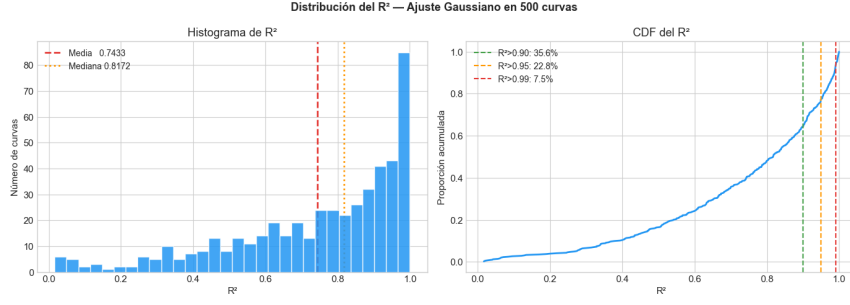


Figura 9: Distribución del  $R^2$  con una sola gaussiana sobre las 500 curvas. La cola larga indica que solo una minoría se ajusta bien.

### 3.2.2. Suma de gaussianas y splines (notebook 02)

La extensión natural es sumar  $N$  campanas:

$$y = c + \sum_{i=1}^N A_i \cdot e^{-\frac{(x-\mu_i)^2}{2\sigma_i^2}}.$$

La inicialización es crítica: se usa `scipy.signal.find_peaks` con prominencia del 5% del rango vertical para detectar picos y proponer valores iniciales. El número máximo de campanas se fija en 8.

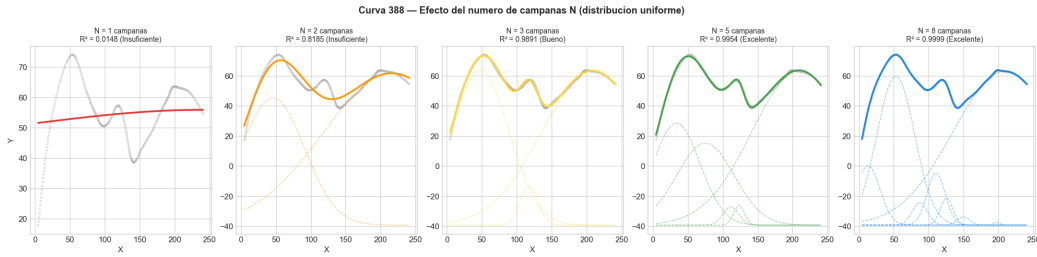


Figura 10: Ajuste con suma de hasta 8 campanas gaussianas: cobertura por componente y reconstrucción global.

Con esa estrategia el  $R^2$  promedio sobre las 500 curvas pasa a 0,9997 con mediana 0,9999.

El mismo notebook compara con splines cúbicos suavizados (`UnivariateSpline`), que funcionan como una familia distinta y obtienen  $R^2$  promedio 0,9994.



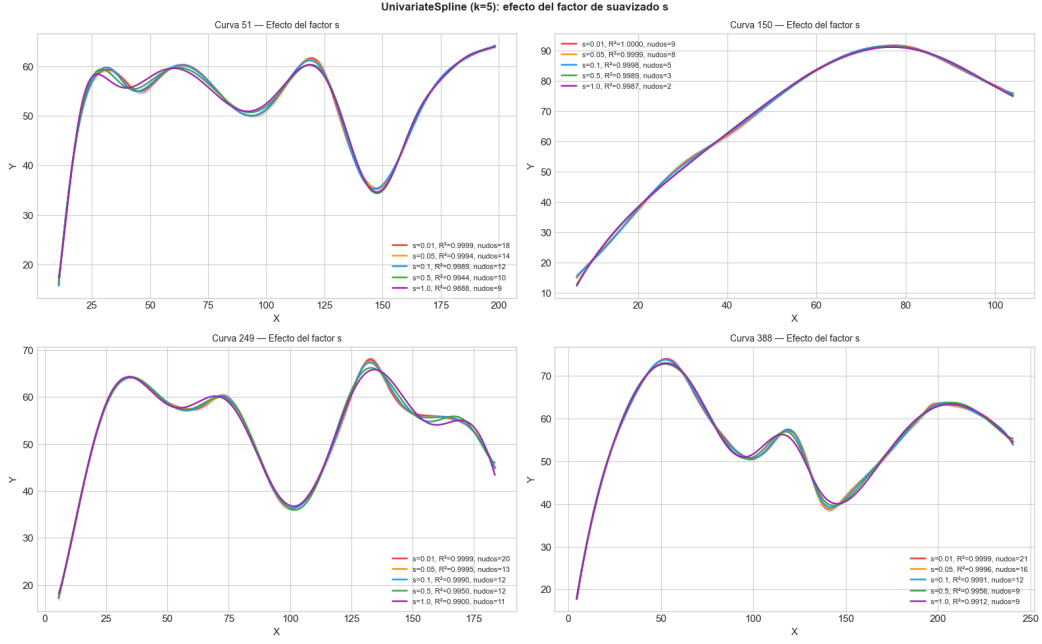


Figura 11: Ajustes spline cúbicos: nodos y reconstrucción.

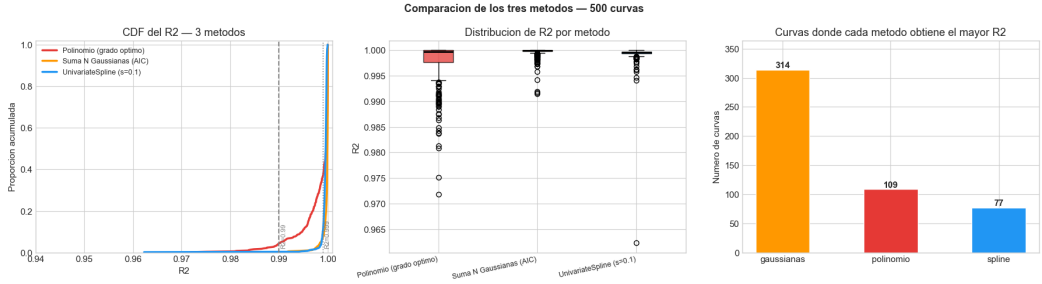


Figura 12: Comparación cabeza a cabeza entre suma de gaussianas y splines.

Las salidas de esta fase son `parametros_gaussianas_por_curva.csv` (28 columnas con  $c$ ,  $A_i$ ,  $\mu_i$ ,  $\sigma_i$  para hasta 8 campanas) y `parametros_spline_por_curva.json`.

#### 4. Fase 3: comparación entre métodos

El script `aproximaciones/comparacion/comparacion_3_metodos.py` construye una tabla de 500 filas con  $R^2$ , RMSE, MAE, error absoluto máximo y número de parámetros para los tres métodos. La columna `mejor_metodo` guarda el ganador por curva (mayor  $R^2$ ).

| Método     | $R^2$ medio | $R^2$ mediano | Curvas donde gana |
|------------|-------------|---------------|-------------------|
| Polinomio  | 0.9981      | 0.9996        | 109 (21.8 %)      |
| Gaussianas | 0.9997      | 0.9999        | 314 (62.8 %)      |
| Spline     | 0.9994      | 0.9996        | 77 (15.4 %)       |

Cuadro 1: Resultados sobre el target de alta precisión. Los tres métodos son excelentes en promedio, pero ninguno gana en todas.

La conclusión es clara: ningún método domina universalmente. La pregunta natural es por qué: ¿qué tipo de curva favorece a cada uno?

## 5. Fase 4: caracterización

El notebook `caracterizacion/caracterizacion_modelos_curvas.ipynb` fusiona las tres tablas anteriores y construye descriptores adicionales:

- `gauss_picos_significativos`: campanas con amplitud mayor al 15 % del máximo.
- `gauss_dispersion_centros`: dispersión espacial ponderada de los centros de campana.
- `gauss_sigma_media_pond`: ancho promedio ponderado de las campanas.
- `indice_compacidad_picos`: picos significativos divididos entre dispersión de centros.
- `bic_promedio` y `r2_std_escalas`: complejidad y variabilidad del polinomio entre escalas.

Tres curvas representativas dan intuición visual del contraste:

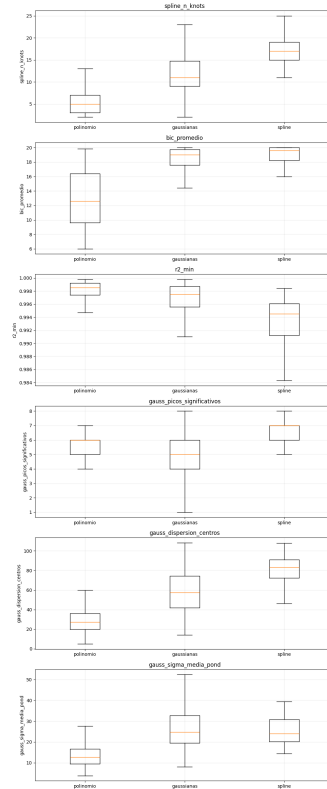


Figura 13: Tres ejemplos de contraste: curva 498 (polinomio gana), curva 1 (gaussianas) y curva 391 (spline).

Para validar qué descriptores discriminan estadísticamente entre los tres grupos se aplican pruebas de Kruskal-Wallis: 12 de 14 variables resultan significativas a  $p < 0,05$ . Después se entrena un árbol de decisión de profundidad 3 que produce reglas legibles, y un Random Forest de 300 árboles para rankear la importancia.

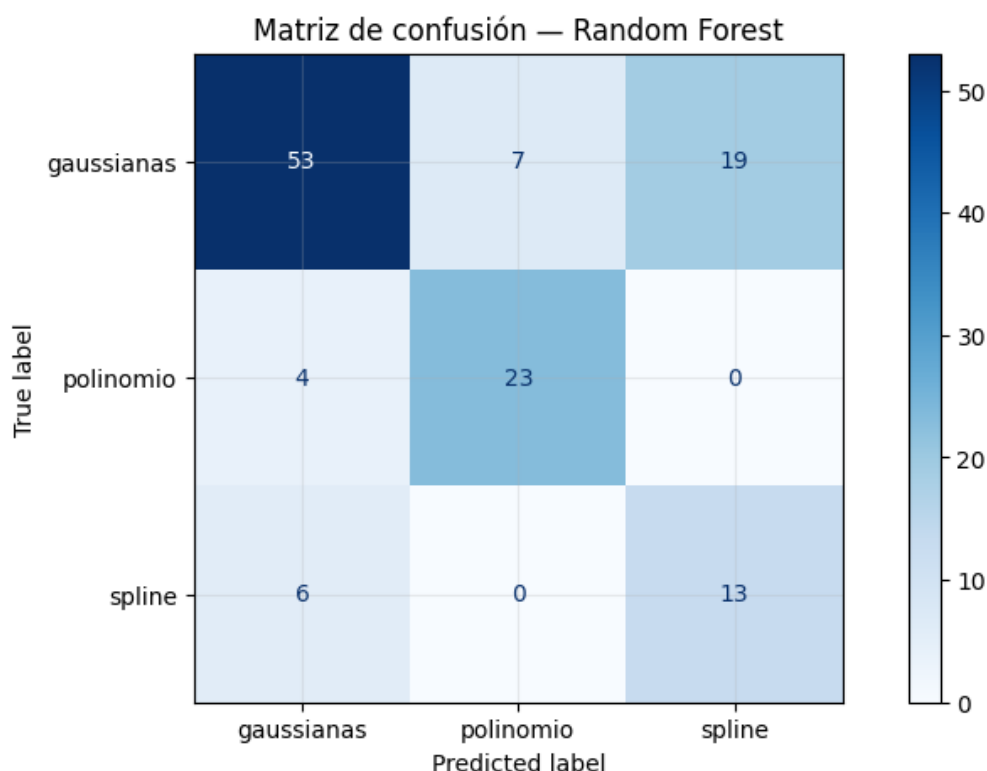


Figura 14: Importancia de variables según Random Forest y reglas extraídas del árbol de decisión.

Las variables más explicativas son `bic_max`, `aic_promedio`, `bic_promedio`, `gauss_sigma_std_pond`, `gauss_sigma_media_pond` y `r2_std_escala`. La conclusión narrativa que sale del análisis es:

- **Polinomio** gana en curvas de estructura global y baja complejidad efectiva. La forma se describe con una sola función sobre todo el dominio. BIC bajo (mediana 12.6), pocos nodos spline necesarios, baja dispersión de centros gaussianos.
- **Gaussianas** gana en curvas con picos bien definidos y estructura multimodal. La forma es compatible con una descomposición en campanas. BIC promedio 19.0, dispersión de centros intermedia, varios picos significativos.
- **Spline** gana en curvas con variación local rica y distribuida. Necesitan flexibilidad por tramos en lugar de un modelo global. Mediana de 17 nodos y la mayor dispersión espacial de centros gaussianos.

El segundo notebook de la carpeta, `analisis_descriptores.ipynb`, profundiza el análisis con ingeniería de features adicional, validación estadística cruzada y experimentos de clustering. Define explícitamente dos modos de trabajo del clasificador: uno con 25 descriptores (incluyendo métricas de ajuste ya calculadas) que alcanza  $F1\text{-macro} = 0,983$ , y otro restringido a 12 descriptores geométricos puros calculables directamente desde datos crudos, que alcanza  $F1\text{-macro} = 0,698$ .

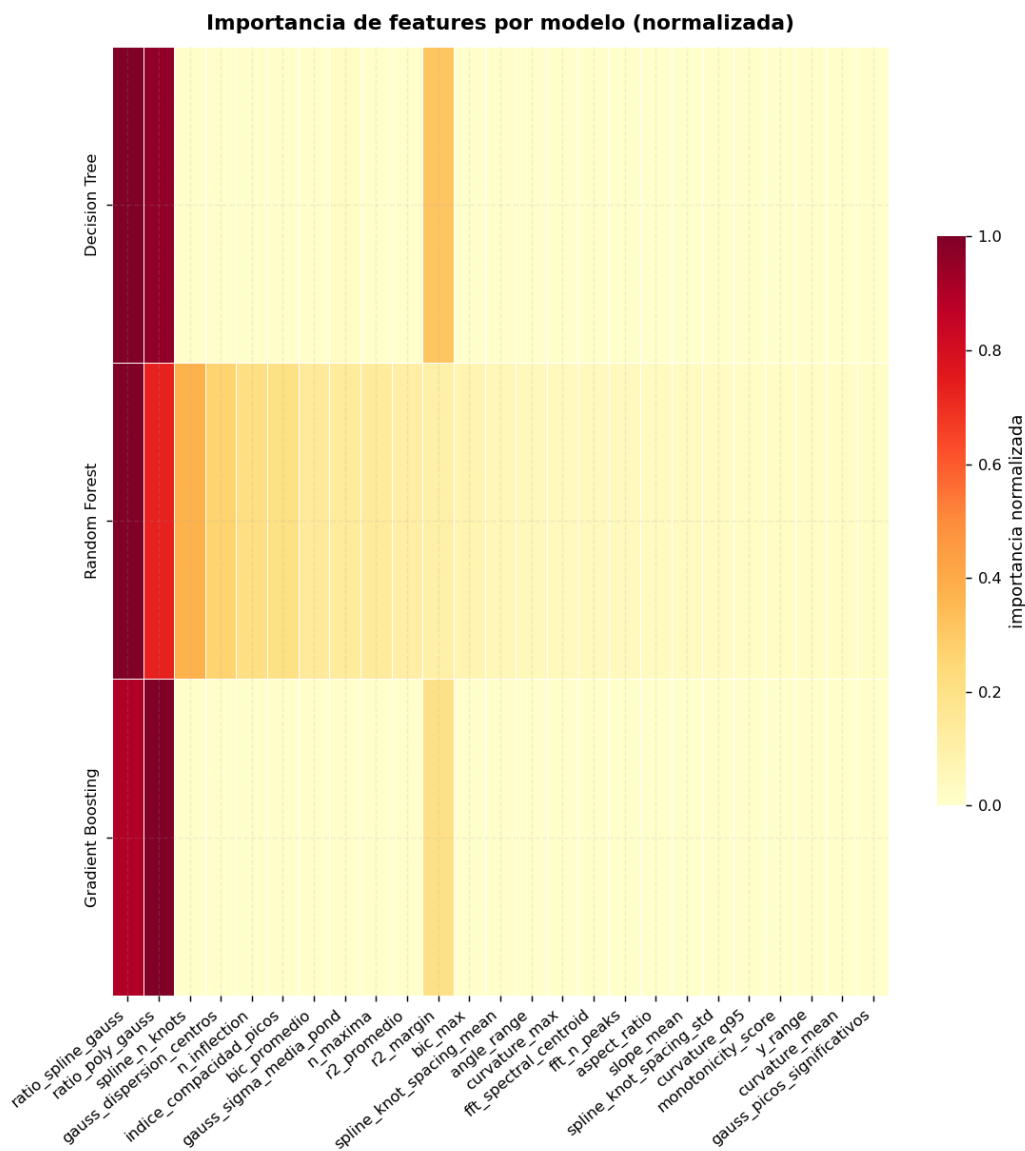


Figura 15: Importancia de features y desempeño comparado en el análisis de descriptores.

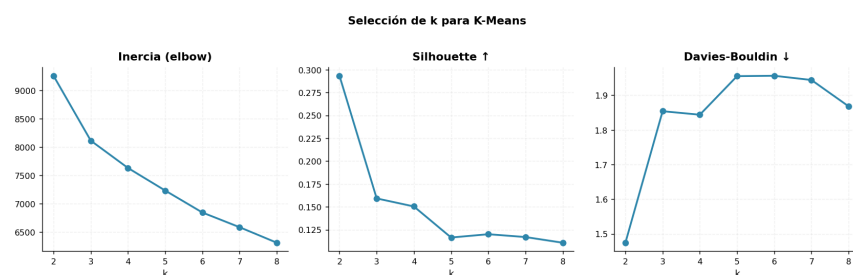


Figura 16: Matrices de confusión del clasificador entre los tres métodos.

## 6. Fase 5: sistema adaptativo

El notebook `sistema_adaptativo/sistema_adaptativo.ipynb` convierte el conocimiento anterior en un sistema operativo. Tiene tres ingredientes:

## 6.1. Selector de método

Se entrena un clasificador (pipeline scikit-learn) sobre los 12 descriptores geométricos calculables desde datos crudos. Se empaqueta como función `recomendar_metodo(ruta)` que recibe una curva y devuelve método óptimo más probabilidad. El modelo serializado se guarda en `modelos/modelo_seleccion_metodo.pkl`.

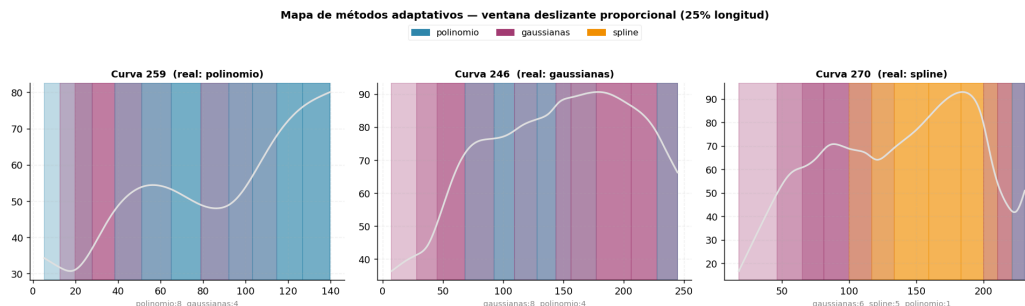


Figura 17: Validación del selector sobre curvas de prueba.

## 6.2. Segmentación adaptativa

En lugar de asignar un único método a toda la curva, se evalúan los tres ajustes en ventanas deslizantes y se elige el mejor localmente. Esto produce un mapa de métodos por zonas, útil para curvas heterogéneas donde el selector global puede fallar.

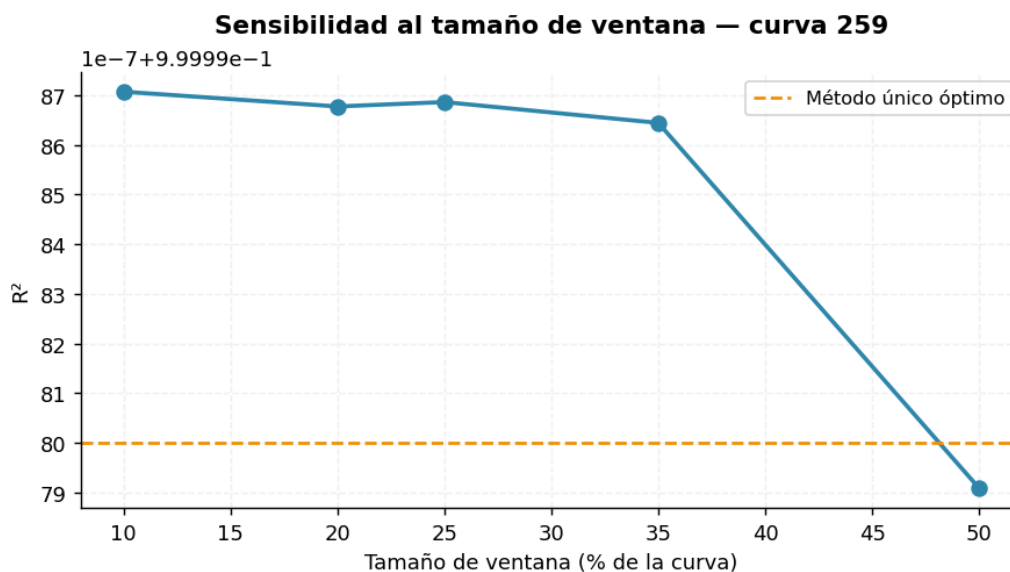


Figura 18: Segmentación adaptativa: mapa de métodos por zonas de la curva.

## 6.3. Benchmark y síntesis

El notebook compara las tres estrategias (selector global, segmentación, ensemble) frente al uso de un único método fijo. Confirma que el sistema adaptativo recupera el  $R^2$  correcto en curvas donde un método único falla, sin penalizar a las que ya funcionaban bien.

## 7. Fase 6: reconstrucción desde píxeles

El último notebook, `reconstruccion_desde_pixeles/reconstruccion_desde_pixeles.ipynb`, pone a prueba el sistema en su escenario más exigente: ya no parte de la curva de alta precisión,

sino de las versiones pixelizadas. La pregunta es si todo el pipeline (descriptores, selector, ajuste) sobrevive al ruido de discretización.

El procedimiento por curva y escala: leer los píxeles, convertir a unidades reales (multiplicar por el factor inverso), pasar por el selector, ajustar con el método elegido y evaluar contra el target.

### 7.1. Caso individual

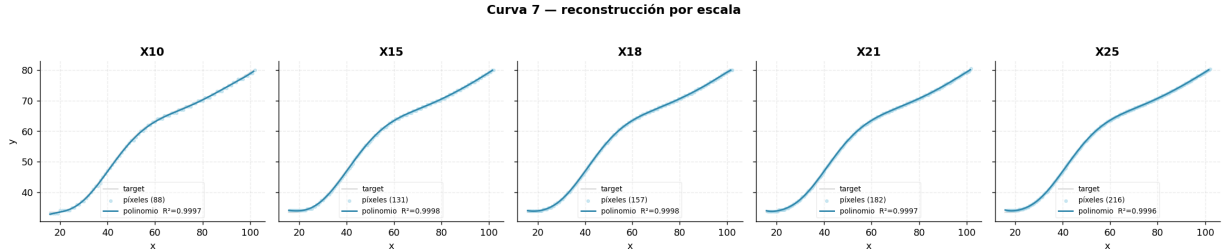


Figura 19: Reconstrucción de la curva 7 en las cinco escalas.

### 7.2. Reconstrucción masiva

Las 500 curvas se procesan en las 5 escalas (2500 reconstrucciones) en aproximadamente 195 segundos. El  $R^2$  medio resulta de **0.952** con mediana **0.992**. La distancia entre ambas es la pista más importante: la mayoría de las curvas se reconstruyen casi perfectas, pero existe una cola de fallos serios (mínimo  $R^2 = -4,74$ ) que arrastra la media.

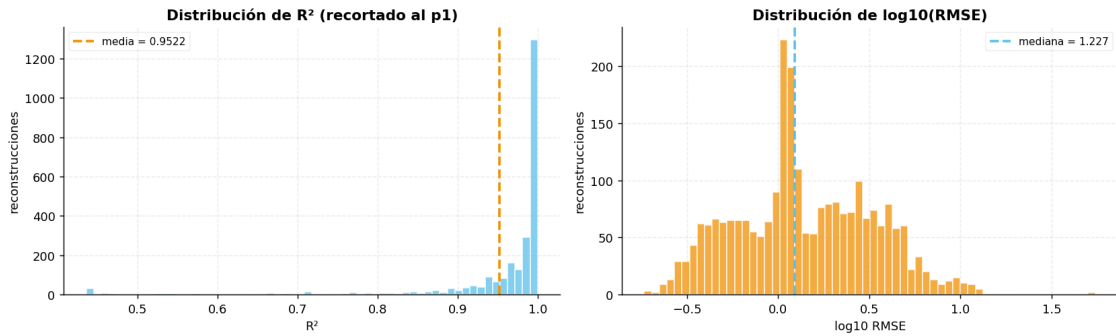


Figura 20: Distribución de  $R^2$ , RMSE y MAE por escala.

Por método elegido por el selector:

| Método     | $R^2$ medio | Veces elegido | % del total |
|------------|-------------|---------------|-------------|
| Polinomio  | 0.9980      | 549           | 22          |
| Spline     | 0.9923      | 424           | 17          |
| Gaussianas | 0.9246      | 1527          | 61          |

Cuadro 2: Cuando el selector escoge polinomio o spline el resultado es casi perfecto. El cuello de botella es que escoge gaussianas el 61 % de las veces, y ahí concentra los fallos.

### 7.3. Análisis de fallos

Definiendo fallo como reconstrucción con  $R^2 < 0,5$ , hay 40 fallos en 2500 reconstrucciones (1.6 %). Todos los fallos ocurren cuando el selector eligió gaussianas; polinomio y spline tienen

tasa de fallo cero. La tasa de fallo además aumenta con la escala (X25 es la peor), porque a más píxeles más picos espurios detecta `find_peaks`, lo que infla descriptores como `fft_n_peaks` y `n_maxima` y empuja al selector hacia gaussianas.

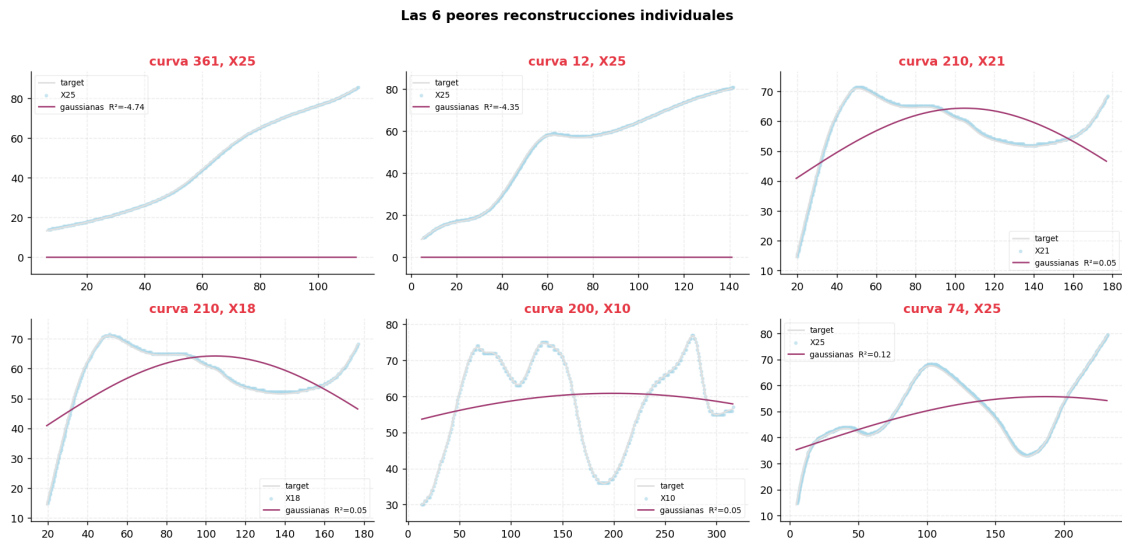


Figura 21: Galería de las peores reconstrucciones individuales: en todas el método elegido es gaussianas.

La conclusión accionable: el selector está entrenado con descriptores calculados sobre el target de alta precisión. Aplicado a píxeles ruidosos sobre-elige gaussianas. Reentrenarlo con descriptores calculados directamente sobre datos pixelizados, suavizar antes de detectar picos y usar spline como fallback en lugar de polinomio de grado 10 son las tres mejoras inmediatas identificadas.

## 7.4. Modo interactivo

El notebook incluye dos modos donde el usuario genera su propia curva, el sistema la pixeliza en las cinco escalas igual que el dataset original y la reconstruye en vivo. El primero es trazo libre con el ratón (requiere `ipympl`) y el segundo usa sliders paramétricos que funcionan en cualquier entorno inline.

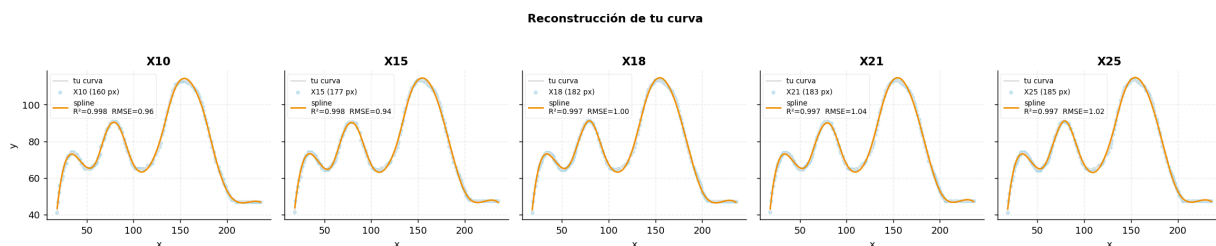


Figura 22: Modo 1 en acción: curva dibujada a mano con múltiples picos y su reconstrucción en las cinco escalas. El selector eligió spline en todos los casos.

## 8. Conclusión global

El proyecto cubrió un ciclo completo de análisis: entender los datos, probar tres familias de modelos, elegir el ganador por curva, explicar por qué gana, construir un sistema que tome esa decisión automáticamente y poner ese sistema a prueba en condiciones realistas (datos pixelizados).

Los hallazgos sustantivos son tres. Primero: las 500 curvas son funciones  $y = f(x)$  bien comportadas y los tres métodos las aproximan con  $R^2$  mediano por encima de 0.999 sobre el target. Se-

gundo: el método óptimo no es universal, depende de la geometría intrínseca de la curva (suavidad global, multimodalidad, variación local) y esa relación es estadísticamente robusta y capturable por un clasificador interpretable. Tercero: el clasificador entrenado sobre datos limpios sufre un sesgo predecible cuando se aplica a píxeles ruidosos, y el camino de mejora es claro y concreto.

---

*Documento generado a partir de los notebooks del repositorio. Todos los porcentajes,  $R^2$ , conteos y figuras provienen directamente de los archivos en `curves/`.*